Deep learning - In a nutshell pt2

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are designed for sequence-based tasks, where the output depends not only on the current input but also on previous inputs. This makes them particularly useful for tasks like time series prediction, natural language processing (NLP), and sequential data modeling.

Key characteristics of RNNs:

- **Recurrent Connections:** RNNs have a loop structure that allows them to "remember" information from previous time steps. Each neuron receives not only the current input but also its previous output, allowing the network to maintain a form of internal memory.
- **Short-term Memory:** Due to their recurrent nature, RNNs maintain some memory of recent inputs. However, this memory fades quickly as the network struggles to retain information over long sequences.
- **Training Challenges:** A major limitation of basic RNNs is the "vanishing gradient problem," where gradients diminish over time during backpropagation through many time steps, making it hard to train networks effectively on long sequences. This led to the development of more sophisticated variants like LSTMs and GRUs.

Code Snippet

```
from keras.models import Sequential
from keras.layers import SimpleRNN, Dense

# Create a sequential model
model = Sequential()

# Add a SimpleRNN layer
# units=50: number of neurons
# input_shape=(timesteps, features): define input shape, e.g., 10 t
# Default Activation for RNNs is Tanh
model.add(SimpleRNN(units=50, input_shape=(10, 1), ))
```

```
# Add a Dense output layer with 1 unit (for regression or binary cl
model.add(Dense(units=1))

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Summary of the model architecture
model.summary()
```

- `SimpleRNN` is the basic RNN layer in Keras.
- `units=50`: The number of neurons (or hidden units) in the RNN layer.
- `input_shape=(10, 1)`: This defines the shape of your input data—10 time steps and 1 feature per time step.
- The output is a single neuron for regression or binary classification, added with the `Dense` layer.
- The model is compiled using the Adam optimizer and the mean squared error (MSE) loss, suitable for regression.

Long Short-Term Memory Networks (LSTMs)

Long Short-Term Memory (LSTM) networks are an improved version of RNNs designed to overcome the problem of retaining long-term dependencies. LSTMs are widely used for time series forecasting, speech recognition, and natural language processing tasks such as machine translation and text generation.

Key characteristics of LSTMs:

- **Memory Cells:** LSTMs contain "memory **cells**" that can store information for long periods of time. These cells are regulated by three gates:
  - **Input Gate:** Decides which **information** from the current **input** should be added to the **memory**.
  - **Forget Gate:** Controls which information should be discarded from the memory.

    - **Output Gate:** Determines what part of the memory should be outputted to the next time step.

- **Long-term Memory:** Unlike basic RNNs, **LSTMs** can learn both short-term and long-term **dependencies**, making them highly effective at capturing **patterns in sequential data** over long time spans.
- **Training Stability:** The gating mechanisms in LSTMs prevent the vanishing gradient problem by ensuring that important information is retained and propagated through the network.

Code Snippet

```
from keras.models import Sequential
from keras.layers import LSTM, Dense

# Create a sequential model
model = Sequential()

# Add an LSTM layer
# units=50: number of memory units
# input_shape=(timesteps, features)
model.add(LSTM(units=50, input_shape=(10, 1)))

# Add a Dense output layer with 1 unit
model.add(Dense(units=1))

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Summary of the model architecture
model.summary()
```

- `LSTM`: This layer introduces the LSTM architecture, allowing the model to capture both short-term and long-term dependencies.
- Similar to the RNN example, the `input_shape=(10, 1)` indicates that we are feeding 10 time steps with 1 feature per step.

- The LSTM layer outputs a single value from the `Dense` layer, which can be used for regression.
- This model is compiled with the Adam optimizer and MSE loss, typically used for time series regression tasks.

Gated Recurrent Units (GRUs)

Gated Recurrent Units (GRUs) are another variant of RNNs that aim to simplify the architecture of LSTMs while retaining their ability to model long-term dependencies. GRUs have fewer parameters than LSTMs, making them computationally faster and more efficient, particularly for certain tasks where long-term memory may not be as crucial.

Key characteristics of GRUs:

- **Gating Mechanism:** GRUs combine the forget and input gates into a single **update gate**. This reduces the **complexity of the architecture** while still maintaining the ability to control the flow of information through the network.
- **Reset Gate:** The reset gate controls how much of the previous state to forget, which allows the network to reset its memory when processing a new sequence element.
- **Efficiency:** GRUs are often preferred when **computational efficiency** is important or when long-term dependencies are not as prominent in the data. In many cases, **GRUs perform similarly to LSTMs but with fewer parameters and faster training times.**

Code Snippet

```
from keras.models import Sequential
from keras.layers import GRU, Dense

# Create a sequential model
model = Sequential()

# Add a GRU layer
# units=50: number of neurons
# input_shape=(timesteps, features)
```

```
model.add(GRU(units=50, input_shape=(10, 1)))

# Add a Dense output layer with 1 unit
model.add(Dense(units=1))

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Summary of the model architecture
model.summary()
```

- `GRU`: This layer introduces the GRU architecture, a simplified alternative to LSTM, capturing long-term dependencies with fewer parameters.
- Like the LSTM and RNN models, `input_shape=(10, 1)` indicates that we are processing sequences with 10 time steps and 1 feature per step.
- The GRU layer is followed by a `Dense` layer with 1 unit for regression or binary classification tasks.
- The Adam optimizer and MSE loss function are used in the compilation step.

Comparison

Feature	RNN	LSTM	GRU
Memory Retention	Short-term	Long-term (via memory cells)	Long-term (simplified)
Gates	None	Input, Forget, Output	Update, Reset
Training	Susceptible to vanishing gradients	Solves vanishing gradient problem	Solves vanishing gradient problem
Complexity	Simple	Complex (more parameters)	Simpler than LSTM (fewer parameters)
Performance	Struggles with long sequences	Great for long sequences	Efficient for medium-length sequences

Key thing to note: Transformers have largely replaced RNNs in the domain of NLP, this means the core use cases of RNNs are **time series prediction** and **anomaly**

detection.

Vanishing and Exploding Gradients

Vanishing Gradients:

- **Vanishing gradients** occur when the gradients (the error signals used to update the weights in a neural network) become very small during backpropagation. This leads to minimal weight updates, which in turn makes it difficult for the model to learn, particularly in deep networks with many layers.
- The problem is especially pronounced in **Recurrent Neural Networks (RNNs)** and other deep architectures, where the gradients diminish as they are propagated back through time or layers. This makes it hard for the network to capture long-term dependencies, as the early layers (or initial time steps in RNNs) stop learning.
- **Symptoms**: Models suffering from vanishing gradients may show slow or stagnant learning, where the error reduces very slowly or not at all, even after many epochs.

Exploding Gradients:

- **Exploding gradients** occur when gradients grow exponentially during backpropagation. This causes extremely large updates to the network weights, which can result in unstable training, leading to large weight values, model divergence, or NaN values in the training process.
- Like vanishing gradients, this problem is more common in **RNNs** and deep networks, where gradients are propagated over many layers or time steps.
- **Symptoms**: Models suffering from exploding gradients often show erratic behavior, such as rapid jumps in the loss function or failure to converge during training.

There’s a lot of terms, here’s a table that might help.

Model Type	Primary Use Cases	Strengths	Limitations
RNN (LSTM, GRU)	Time series prediction, NLP (historically), Speech recognition, Anomaly detection	- Handles sequential data well (e.g., time series, speech). - Captures temporal dependencies. - LSTMs/GRUs solve the vanishing gradient problem.	- Struggles with long-range dependencies without enhancements. - Sequential processing makes them slow for large datasets.
Transformers	NLP (current state-of-the-art), Machine translation, Text generation, Time series prediction, Anomaly detection	- Captures long-range dependencies. - Parallelizable, faster training. - Self-attention improves performance on large datasets. - Dominates NLP tasks (BERT, GPT).	- Requires large datasets. - Computationally expensive. - May be overkill for simple tasks or short sequences.
CNN (Convolutional Neural Network)	Image classification, Object detection, Image segmentation, Video analysis, Time series (less common)	- Excellent for spatial data (e.g., images, videos). - Recognizes local patterns effectively (edges, textures). - Computationally efficient due to parameter sharing.	- Not designed for handling temporal dependencies (e.g., time series). - Typically limited to 2D or 3D spatial data.
Autoencoders	Anomaly detection, Dimensionality reduction, Data compression, Image denoising	- Effective for unsupervised learning tasks.- Captures data structure for dimensionality reduction and anomaly detection. - Used for data reconstruction tasks (e.g., denoising).	- Limited in handling long-term temporal dependencies. - Often requires a second model or method for prediction tasks.

DNN (Deep Neural Network)	General-purpose, Classification, Regression, Time series forecasting (with feature engineering), Anomaly detection	- Flexible and general-purpose. - Can model complex, non-linear relationships.- Useful for classification and regression.	- Does not inherently capture sequential or temporal dependencies. - Requires large datasets for training. - Susceptible to overfitting without proper regularization.

Table 1: Optimizer breakdown

Optimizer	Pros	Cons	Use Cases
Adam	Combines AdaGrad and RMSProp benefits.	Requires more memory for storing moving averages.	General-purpose, NLP, computer vision.
Stochastic Gradient Descent (SGD)	Efficient computation with mini-batches.	Noisy updates can lead to less stable convergence.	Large-scale machine learning, online learning.
AdaGrad	Adapts learning rate for each parameter.	Learning rate may decay too much over time.	Sparse data, natural language processing.
Mini-batch SGD	Balances speed and stability.	Still noisier than full-batch.	General-purpose, deep learning.
Gradient Descent	Simple and easy to understand.	Slow convergence on large datasets.	Small datasets, linear regression.
Momentum	Accelerates convergence by using past gradients.	Requires tuning of momentum parameter.	Deep networks, image classification.
RMSProp	Maintains a moving average of squared gradients.	Requires careful tuning of the decay rate.	Recurrent neural networks, deep learning.
AdaDelta	No need to manually set learning rate.	Can be computationally expensive.	Complex models with varying gradients.

Table 2: Hidden Layer Activation Functions

Function	Pros	Cons	Use Cases	Neural Network Type
Tanh	Zero-centered, stronger gradients than sigmoid.	Vanishing gradient for large inputs.	Binary classification, RNNs.	Hidden layers in RNNs, simple networks.
ReLU	Efficient computation, mitigates vanishing gradient.	Dead neurons (outputs stuck at zero).	General-purpose, CNNs, DNNs.	Hidden layers in CNNs and DNNs.
Leaky ReLU	Solves the dead neuron problem.	Slightly more complex computation.	General-purpose, prevents dead neurons.	Hidden layers in CNNs, DNNs.
ELU	Smooth gradient, faster convergence for some models.	Computationally more expensive than ReLU.	General-purpose, better for deep networks.	Hidden layers in deep networks.
Sigmoid	Not commonly used in modern networks as a hidden layer	Vanishing gradient, non-zero-centered outputs.	Cases where binary decision-making is required at intermediate steps.	Output layers in simple networks

Note: While **sigmoid** is not commonly used in hidden layers of modern deep networks, it still has applications in specific cases where binary outputs or probabilistic interpretations are needed. However, in most cases, other activations like **ReLU**, **Leaky ReLU**, or **Tanh** tend to be favored in hidden layers due to their efficiency and ability to mitigate issues like vanishing gradients.

Table 3: Output Layer Activation Functions

Function	Pros	Cons	Use Cases	Neural Network Type
Sigmoid	Smooth gradient, outputs between 0 and 1.	Vanishing gradient, non-zero-centered outputs.	Binary classification tasks.	Output layers in simple networks
Softmax	Provides probabilistic interpretation of outputs.	Not suitable for hidden layers due to normalization.	Multi-class classification.	Output layers in classification networks.
Linear	Direct mapping between inputs and outputs.	No non-linearity, limited in complex tasks.	Regression tasks.	Output layers in regression networks.

When to Use Tanh vs. ReLU in RNNs

Tanh

- **Use Case**: Often used in recurrent neural networks (RNNs), especially in the hidden layers.
- **Pros**:
  - Outputs are zero-centered, which helps with optimization.
  - Suitable for capturing complex patterns in sequential data.
- **Cons**:
  - Can suffer from vanishing gradients, especially in very deep networks.

ReLU

- **Use Case**: Commonly used in feedforward networks and can be applied in RNNs for certain tasks.
- **Pros**:
  - Efficient computation and helps mitigate vanishing gradient problems.
  - Allows for faster convergence.

- **Cons**:
    - Can suffer from the "dying ReLU" problem, where neurons may stop activating.

Note

A **dying neuron** refers to a situation in neural networks where a neuron (or unit) stops updating its weights during training and becomes permanently inactive, meaning it always outputs zero for any input. This issue is most commonly associated with the **ReLU (Rectified Linear Unit)** activation function.

Summary

- **Tanh** is preferred in RNNs when capturing intricate patterns and ensuring zero-centered outputs are important.
- **ReLU** is chosen for its efficiency and when faster training times are needed, though care must be taken to handle potential dead neurons.